

API Gateway: A Self-Service Rate-Limiting Layer for Web APIs

Project Proposal for UCS503P
Submitted to: Dr. Paramveer Sidhu

Thapar Institute of Engineering and Technology

Sarthak Jagota - 1024030753

Palak Mahajan - 1024030755

Aayush Kumar Singh - 1024030758

August 8, 2026

1 Higher-order goal

... secondary framing

This project aims to improve the *resilience and cost-predictability of networked services* by making traffic control available to developers who cannot afford, or do not wish to operate, enterprise API-management infrastructure. While traffic shaping is a broad concern spanning cloud economics and denial-of-service resistance, the immediate engineering objective is narrower and concrete: to translate *fair, enforceable request budgeting* into a deployable software layer that any API owner can place in front of an existing service without modifying that service.

2 Time-to-value

... primary heuristic

- The smallest useful product is a proxy that forwards a request and returns HTTP 429 when a per-client budget is exhausted. This is demonstrable in the first iteration and is independently valuable even with no dashboard, no persistence and a single algorithm.
- We will prioritise fast feedback over completeness by using weekly iterations with CI-backed automation from the outset, so that each increment is releasable to staging rather than accumulating on a branch.
- The work splits into three parallel workstreams — gateway and rate-limiting logic, persistence and authentication, and the analytics frontend — allowing the team to produce measurable increments concurrently rather than serially.
- Success in iteration one is defined by measurement, not by feature count: enforcement must be observably correct under concurrent load before any feature is layered on top of it.

3 Problem Statement

An HTTP API, by default, has no notion of a per-client budget. Every client is free to issue requests as fast as it can open connections, and the server discovers the consequences only after they occur. This produces four recurring failure modes:

- **Resource exhaustion:** a single misbehaving client—whether a retry loop caused by a bug, a scraper, or a deliberate abuser—can saturate connection pools, database connections or worker threads, and degrade service for every other client.
- **Uncontrolled cost:** where the API is backed by metered resources (managed databases, third-party APIs, egress bandwidth), unbounded request volume translates directly into unbounded expenditure, discovered only at billing time.
- **No fairness guarantee:** without per-client accounting, a well-behaved consumer and an abusive one are indistinguishable to the server, so capacity is allocated by aggression rather than by policy.
- **High cost of the remedy:** the standard solution—a rate-limiting gateway between clients and the origin—exists in mature form (Kong, Cloudflare, AWS API Gateway) but is either operationally heavy or commercially priced. A developer wishing to protect a single API must otherwise build the counting, storage and enforcement logic themselves.

Why this matters now (contextual relevance): Deployment on shared platform-as-a-service tiers has made small APIs cheap to publish and correspondingly easy to overwhelm; the marginal cost of a request has moved from the developer’s fixed server bill to a metered per-request charge. The absence of a lightweight, self-service traffic-control layer is therefore a present and practical gap rather than a hypothetical one.

4 Proposed Solution

... *Software Proposition*

4.1 Overview

We propose **API Gateway**, a web-based API protection layer that sits as a reverse proxy between a client and a target API. Its components are:

- **Gateway/proxy layer:** intercepts every inbound request, evaluates it against the applicable policy, forwards permitted requests to the registered origin and rejects the remainder with HTTP 429 (**Too Many Requests**).
- **Self-service registration portal:** an authenticated user registers a target API URL, sets a request budget (requests per unit time), and selects the enforcement algorithm.
- **Distributed counting substrate:** per-client counters held in Redis and manipulated through Bucket4j, so that the limit remains correct when several gateway instances serve the same API.
- **Analytics dashboard:** a React interface showing allowed versus blocked volume, traffic over time, and a per-API breakdown.
- **Request log:** a queryable table of recent requests with outcome, timestamp and client identity, forming the evidential basis for both the dashboard and our evaluation metrics.

4.2 Core workflow

1. A developer signs up, authenticates, and registers a target API, receiving a gateway endpoint and an API key.
2. A client issues a request to that gateway endpoint bearing the key.
3. The gateway resolves the policy for (client, API), and performs an atomic consume-or-reject operation against the Redis-backed counter.

4. If a token is available the request is proxied to the origin and the origin's response is relayed verbatim; otherwise the gateway short-circuits with HTTP 429 and the applicable `Retry-After` hint.
5. In both cases an outcome record is written to PostgreSQL.
6. The dashboard reads aggregated records over REST and renders them.

4.3 Operational constraints for a course-project setting

- The gateway must add only bounded latency to the proxied path; a correct limiter that doubles response time is not a usable one.
- We avoid dependence on novel research: token bucket and sliding window are well-specified, published algorithms, and our contribution is their correct distributed implementation and instrumentation, not their invention.
- The whole system must deploy on free or low-cost managed tiers (application host, managed Redis, managed PostgreSQL) so that the pilot is reproducible by the evaluator without provisioning cost.
- The dashboard must remain usable on low-to-mid range devices through responsive design and server-side aggregation rather than shipping raw logs to the browser.

5 Solution Approach

...Engineering focus

This is an engineering course project. Accordingly the emphasis falls on correctness under concurrency, clean separation of concerns, and measurable behaviour, rather than on algorithmic novelty.

5.1 Rate-limiting strategy

...non-research

Enforcement uses two standard algorithms, both provided by `Bucket4j` over a Redis backend:

- **Token bucket:** a bucket of capacity C refilled at rate r ; each request consumes one token, and a request arriving at an empty bucket is rejected. This permits controlled bursts up to C while bounding the long-run average at r .
- **Sliding window:** the count of requests within a moving interval of width W is compared against a threshold N , giving smoother enforcement at the cost of more state per client.
- **Correctness under distribution:** counter updates are performed as atomic server-side operations in Redis, so that two gateway instances handling simultaneous requests from the same client cannot both observe the same available token. This—not the arithmetic of either algorithm—is the substantive engineering problem.
- **Failure policy:** the behaviour when the counter store is unreachable is an explicit design decision (fail-open, preserving availability, versus fail-closed, preserving protection). We will implement one, document the rationale, and test it.

5.2 Web architecture

...web-based solution

A three-tier architecture with an additional in-memory tier for counters:

- **Frontend:** React single-page application with `Recharts` for visualisation; communicates with the backend exclusively over REST.

- **Gateway/Backend:** Spring Boot with Spring Cloud Gateway for the proxy path; Spring Security with JWT for authentication; Spring Data JPA for persistence.
- **Counter store:** Redis, holding only ephemeral per-client counter state.
- **Database:** PostgreSQL, holding users, registered APIs, policies and request logs.

The request path is summarised below.

```

Client
↓
API Gateway (Spring Cloud Gateway)
↓ consume-or-reject
Bucket4j + Redis (distributed counter)
↓ if allowed
Target API → response relayed to client
↓ always
Request log written → PostgreSQL (Spring Data JPA)
↓
React Dashboard (REST) → charts and logs

```

5.3 Technology selection

Layer	Technology
Gateway / Backend	Spring Boot, Spring Cloud Gateway
Rate limiting	Bucket4j
Counter store	Redis
Primary database	PostgreSQL (Spring Data JPA)
Authentication	Spring Security + JWT
Frontend	React + Recharts
API testing	Postman; JUnit for automated tests
Deployment	Render/Railway (backend, Redis, PostgreSQL); Vercel (frontend)

5.4 CI/CD and fast delivery enabler

CI/CD will be used to:

- build the backend and run unit and integration tests on every change;
- run the load-and-enforcement test suite, so that a regression in limiting accuracy fails the build rather than reaching staging;
- produce repeatable deployment artifacts (container images) for staging;
- enable safe and frequent release of incremental features.

6 Evaluation Criterion

... measurable and attributable

Success is defined by instrumented measurement on the deployed system, not by feature presence.

6.1 Primary evaluation metric

Enforcement accuracy under concurrent load (EA): for a configured limit of N requests per window, the number of requests actually admitted to the origin when $M \gg N$ concurrent requests are issued from a single client identity across all running gateway instances.

- **Target:** admitted count within $\pm 2\%$ of N , with two gateway instances active, over a driver issuing at least $10N$ requests.
- **Attribution:** computed by comparing the load driver's own tally of HTTP 200 versus HTTP 429 responses against the count of forwarded requests recorded at the origin. Over-admission indicates a race in the counter path; under-admission indicates an over-strict window boundary. Both are attributable to the limiter, not to the network.

6.2 Secondary evaluation metrics

- **Added latency (p50/p95):** difference in response time between a request routed through the API Gateway and the same request issued directly to the origin. Target: p95 overhead ≤ 50 ms on the pilot deployment.
- **Sustained throughput:** requests per second the gateway sustains before p95 latency degrades beyond the above threshold, reported per instance count to evidence horizontal scaling.
- **Log completeness:** proportion of driver-issued requests appearing in the PostgreSQL log with the correct outcome. Target: $\geq 99.9\%$, since the dashboard's credibility rests on this.
- **Dashboard freshness:** lag between a request occurring and its appearance in dashboard aggregates.
- **Reliability:** availability of the staging gateway for proxy and login operations during the pilot window (target $\geq 99\%$ uptime).

6.3 Pilot validation plan

... high-level

- Register a small set of representative target APIs (a deliberately slow endpoint, a fast endpoint, and a public third-party API) to exercise differing latency profiles.
- Drive load using a scripted client at controlled concurrency levels, repeating each run with one and with two gateway instances to demonstrate that distributed counting holds.
- Recruit 5–10 developer users to register their own API and report clarity of setup and of the dashboard on a 1–5 scale, providing a usability counterweight to the purely quantitative metrics.

7 Scalability

... with foresight

The proposition should scale in both theoretical and practical senses.

1. **Scalable (theoretically):** the design admits growth in request volume, registered APIs and concurrent dashboard users by:
 - keeping the gateway *stateless*, with all mutable limiting state externalised to Redis, so instances may be added or removed freely behind a load balancer;
 - performing counter updates as single atomic operations, so coordination cost does not grow with instance count;

- writing request logs asynchronously off the request path, so that persistence latency does not enter the client’s critical path;
 - indexing log queries on (`api_id`, `timestamp`) and serving the dashboard from pre-aggregated summaries with pagination rather than raw scans.
2. **Operationally deployable (practically):** the system runs on commodity managed infrastructure:
- managed Redis and managed PostgreSQL instances;
 - containerised deployment of the gateway, permitting horizontal replication;
 - static hosting of the React frontend on a CDN-backed platform;
 - a CI/CD pipeline targeting staging and then production.

8 Engine availability heuristic

A mature set of “engines” exists for every layer of this proposition, which is what makes the scope realistic within a course timeline:

- Spring Boot and Spring Cloud Gateway for the reverse-proxy and REST layers.
- Bucket4j as a maintained implementation of the token-bucket and window algorithms, with an existing Redis integration.
- Redis for atomic, low-latency counter operations.
- PostgreSQL with Spring Data JPA for durable relational storage.
- Spring Security with JWT for token-based authentication.
- React with Recharts for the dashboard and its visualisations.
- JUnit and Testcontainers for integration tests against real Redis and PostgreSQL instances.
- A CI runner (GitHub Actions) and free-tier staging targets.

We use these mature components deliberately, so that our engineering effort is spent on correct distributed enforcement, instrumentation and measurement, rather than on reimplementing infrastructure.

9 Project scope and deliverables

... iteration-friendly

9.1 Initial deliverable

... first iteration

- Working reverse proxy: request forwarded to a configured target API and response relayed.
- Token-bucket enforcement backed by Redis, returning HTTP 429 with `Retry-After` on rejection.
- User signup and login with JWT; API registration with a configurable limit.
- Request logging to PostgreSQL with timestamps and outcome, sufficient to compute the primary metric.
- CI: build, backend unit tests, and linting on every push.
- CD to staging as a single pipeline job.

9.2 Subsequent deliverables

- Sliding-window algorithm as a selectable policy per API.
- React analytics dashboard: allowed versus blocked counts, traffic over time, per-API breakdown.
- Request log view with filtering and pagination.
- Multi-instance validation run demonstrating enforcement accuracy across two gateways.
- (*Stretch*) Request replay: re-issue a specific blocked or failed request from the log.
- (*Stretch*) Live side-by-side comparison of the two algorithms under identical load.

10 Risks and mitigations

- **Counter store as a single point of failure:** Redis unavailability would disable enforcement. *Mitigation:* define and implement an explicit fail-open/fail-closed policy, surface the degraded state in the dashboard, and test the failure path in CI.
- **Latency budget:** an additional network hop plus a counter lookup could exceed the acceptable overhead. *Mitigation:* co-locate gateway and Redis in the same region, keep logging off the request path, and treat p95 overhead as a build-gating metric.
- **Race conditions in distributed counting:** the defining correctness risk of the project. *Mitigation:* rely on atomic server-side operations rather than read-modify-write, and make multi-instance concurrent load tests part of the automated suite from iteration one.
- **Credential and secret handling:** JWT signing keys and registered-API credentials are sensitive. *Mitigation:* store secrets in platform-managed environment configuration, never in the repository; store only what is needed and hash user credentials.
- **Proxy abuse:** an open proxy can be misused to reach arbitrary hosts. *Mitigation:* permit forwarding only to targets registered by an authenticated owner, and validate target URLs on registration.
- **Scope creep from stretch features:** replay and live comparison are attractive but non-essential. *Mitigation:* they are explicitly scheduled last and gated on the primary metric being met.
- **Free-tier deployment limits:** cold starts and connection caps on free tiers can distort latency measurements. *Mitigation:* warm the service before measurement runs and report the deployment tier alongside every figure.

11 Summary

This project proposes API Gateway, a deployable, self-service rate-limiting gateway that lets a developer place enforceable traffic control in front of an existing API without modifying it. It meets the course criteria by emphasising time-to-value—a useful proxy-plus-limiter is releasable in the first iteration—by using CI/CD to support frequent safe releases, and by defining evaluation in measurable, attributable terms, principally enforcement accuracy under concurrent load with added latency as a bounding constraint. It relies throughout on mature components and published algorithms, keeping the engineering contribution in correct distributed implementation, instrumentation and scalability readiness rather than in research novelty.